Fundamental 1팀 스터디 / 딥 러닝 파이토치 교과서

05
소프트맥스 회귀

셋 이상의 선택지 중 하나를 고르는 문제를, 선택지마다 확률을 매겨서 푸는 방법을 배웁니다.

오늘 다루는 것

1. 원-핫 인코딩
2. 소프트맥스 회귀 이해하기
3. 소프트맥스 회귀 다양한 방법으로 구현하기
4. 소프트맥스 회귀로 MNIST 데이터 분류하기

끝나고 나면 할 수 있는 것

1. 정답을 원-핫 벡터로 표현하는 이유를 설명할 수 있다.
2. 소프트맥스와 크로스 엔트로피를 작은 숫자로 직접 계산할 수 있다.
3. 손글씨 숫자 10개를 분류하는 소프트맥스 회귀를 학습시킬 수 있다.

2026년 10월 6일 화요일 (10/6 자료 3/3)
발표자 설명, 코랩 실습, 퀴즈 돌아가며 풀기 순서로 진행합니다. 막힌 질문은 디스코드에 남겨주세요.

05

소프트맥스 회귀

04장은 합격/불합격처럼 둘 중 하나를 골랐습니다. 하지만 붓꽃 품종은 3가지, 손글씨 숫자는 10가지, 문장 속 다음 단어는 수만 가지입니다. 소프트맥스 회귀는 선택지마다 점수를 내고, 그 점수를 합이 1인 확률로 바꿔 가장 그럴듯한 것을 고릅니다. 이 "점수 → 소프트맥스 → 확률" 구조는 GPT가 다음 단어를 고를 때도 똑같이 쓰입니다.

왜 배우나: 오늘의 학습 경로



정답을 벡터로 쓴다
원-핫 인코딩

선택지마다 점수를 낸다
Wx + b (출력 여러 개)

점수를 확률로 바꾼다
소프트맥스

정답 확률로 벌준다
크로스 엔트로피

여러 개 중 하나 맞히기
다중 클래스 분류

05-01, 05-02에서 원리를 익히고, 05-03에서 여러 방법으로 구현해 보며, 05-04에서 MNIST 손글씨를 분류합니다.

05-01 원-핫 인코딩

핵심 한 줄

원-핫 인코딩은 선택지 수만큼 칸을 만들고 정답 자리만 1, 나머지는 0으로 표시하는 방법이다.

왜 배우나

여러 개 중 하나를 맞히는 모델은 선택지마다 확률을 냅니다. 그 확률과 정답을 비교하려면 정답도 같은 모양으로 적어야 합니다. 원-핫 벡터는 "정답 칸만 확률 1"인 모양이라 모델의 출력과 바로 비교할 수 있습니다. 그리고 선택지에 번호를 그냥 붙이면 생기는 문제도 막아 줍니다.

1) 만드는 법

선택지가 강아지, 고양이, 냉장고라면 먼저 번호(0, 1, 2)를 붙이고, 길이 3짜리 벡터에서 자기 번호 칸만 1로 칠합니다. 강아지 = [1, 0, 0], 고양이 = [0, 1, 0], 냉장고 = [0, 0, 1]입니다.

2) 왜 번호를 그대로 쓰지 않나

번호 0, 1, 2를 그대로 쓰면 "냉장고(2)와 강아지(0)의 차이는 2, 고양이(1)와 강아지의 차이는 1"이라는, 원래 없던 **순서와 거리**가 생깁니다. 원-핫 벡터는 모든 쌍의 거리가 똑같아서 이런 엉뚱한 관계를 만들지 않습니다. 다만 같은 이유로 "강아지와 고양이가 비슷하다"는 관계도 표현하지 못합니다. 이 한계는 12장의 워드 임베딩에서 해결합니다.

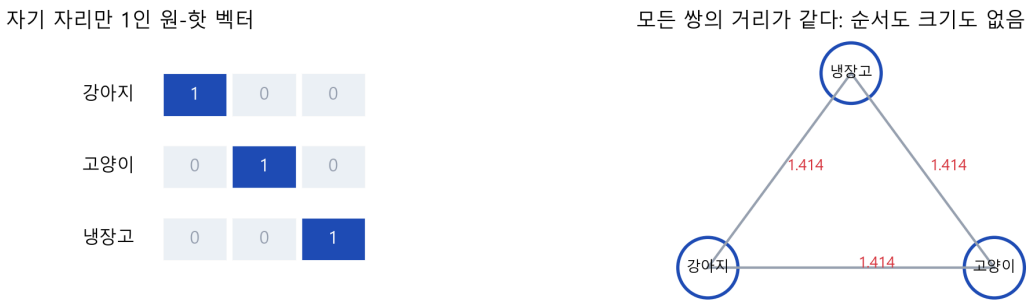


그림 1 왼쪽: 세 선택지의 원-핫 벡터. 오른쪽: 어느 두 선택지 사이의 거리도 똑같이 1.414다

비유

투표 용지의 도장 칸

투표 용지에는 후보마다 칸이 있고, 한 칸에만 도장을 찍습니다. 기호 1번과 3번이 "2만큼 떨어져 있다"고 생각하는 사람은 없습니다. 어느 칸에 찍혔는지만 중요합니다.

후보 칸들 = 벡터의 자리 (선택지 수만큼)
도장 찍힌 칸 = 1인 자리 (정답)
기호 번호 = 인덱스 (순서 의미 없음)

실습 1 원-핫 벡터 만들고 거리 비교하기

```python
import torch
import torch.nn.functional as F

vocab = ["강아지", "고양이", "냉장고"]                # 고를 수 있는 선택지 3개
idx = {w: i for i, w in enumerate(vocab)}           # 강아지=0, 고양이=1, 냉장고=2
print("번호:", idx)

onehot = F.one_hot(torch.tensor([0, 1, 2]), num_classes=3).float()
for w, v in zip(vocab, onehot):
    print(f"{w}: {v.tolist()}")                     # 자기 자리만 1, 나머지는 0

# 번호를 그대로 쓰면 "냉장고(2)가 강아지(0)보다 고양이(1)에서 더 멀다" 같은 엉뚱한 관계가 생긴다
print("번호 차이  강아지-고양이:", abs(0 - 1), "  강아지-냉장고:", abs(0 - 2))
# 원-핫은 모든 쌍의 거리가 같다: 선택지 사이에 순서나 크기가 없다
d = lambda a, b: round(torch.dist(onehot[a], onehot[b]).item(), 3)
print("원-핫 거리  강아지-고양이:", d(0, 1), "  강아지-냉장고:", d(0, 2), "  고양이-냉장고:", d(1, 2))
```

실행 결과

```
번호: {'강아지': 0, '고양이': 1, '냉장고': 2}
강아지: [1.0, 0.0, 0.0]
고양이: [0.0, 1.0, 0.0]
냉장고: [0.0, 0.0, 1.0]
번호 차이  강아지-고양이: 1   강아지-냉장고: 2
원-핫 거리  강아지-고양이: 1.414   강아지-냉장고: 1.414   고양이-냉장고: 1.414
```

직접 해 보기 선택지에 "책상"을 추가해 4개로 만들면 원-핫 벡터의 길이는 몇이 되나요? 선택지가 1만 개라면 어떤 문제가 생길까요?

05-01 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 원-핫 벡터는 정답 자리만 1이고 나머지는 0이다. (O / X) 이름 _____

2 원-핫 벡터는 "강아지와 고양이가 비슷하다"는 관계를 잘 표현한다. (O / X) 이름 _____

3 선택지가 5개일 때 원-핫 벡터의 길이는? 이름 _____

① 1 ② 4 ③ 5 ④ 10

4 번호를 그대로 쓰면 생기는 문제는? 이름 _____

① 원래 없던 순서와 거리가 생긴다 ② 계산이 안 된다 ③ 확률이 1을 넘는다

5 실습에서 고양이의 원-핫 벡터는 [_____]이다. 이름 _____

6 실습에서 모든 쌍의 원-핫 거리는 _____로 같았다. 이름 _____

7 투표 용지 비유로 원-핫 인코딩을 설명해 보세요. 이름 _____

원문: 05-01 원-핫 인코딩 — https://wikidocs.net/59678

05-02 소프트맥스 회귀 이해하기

핵심 한 줄

소프트맥스 회귀는 선택지마다 점수를 내고, 소프트맥스로 합이 1인 확률로 바꾼 뒤, 크로스 엔트로피로 정답 확률이 커지도록 학습한다.

왜 배우나

04장의 시그모이드는 "합격일 확률" 하나만 냅니다. 선택지가 셋 이상이면 확률이 여러 개 필요하고, 그 합이 1이어야 "셋 중 어느 것일 가능성이 가장 큰가"를 말할 수 있습니다. 이 일을 하는 것이 소프트맥스이고, 그에 맞는 손실이 크로스 엔트로피입니다.

1) 소프트맥스 함수

붓꽃 한 송이를 보고 모델이 품종별 점수 [2.0, 1.0, 0.1]을 냈다고 합시다. 점수는 크기도 제각각이고 합도 1이 아닙니다. 소프트맥스는 각 점수에 e를 거듭제곱해 모두 양수로 만든 뒤, 전체 합으로 나눠 확률로 바꿉니다.

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}} = \frac{e^{z_i}}{e^{z_1} + e^{z_2} + \cdots + e^{z_k}}$$

말로 하면: z_i는 i번째 선택지의 점수, k는 선택지 수. 내 점수의 몫 ÷ 모든 점수의 몫의 합. 점수가 클수록 확률이 커지고, 확률의 합은 항상 1입니다.

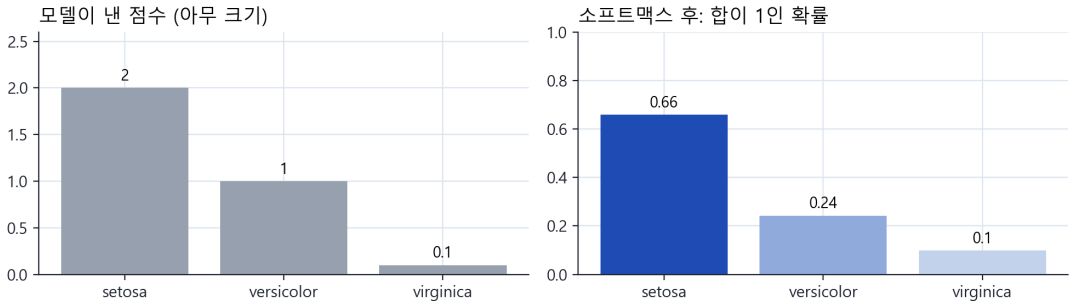


그림 2 모델의 점수(왼쪽)가 소프트맥스를 거쳐 합이 1인 확률(오른쪽)이 된다. 순서는 그대로, 크기만 확률로 바뀐다

2) 모델 구조: 출력이 여러 개인 Wx + b

꽃받침 길이·너비, 꽃잎 길이·너비라는 특성 4개로 품종 3개를 맞힌다면, 가중치는 (4 × 3) 행렬이 됩니다. 샘플 5개를 한 번에 넣으면 (5 × 4) @ (4 × 3) + b = (5 × 3), 즉 샘플마다 점수 3개가 나옵니다. 여기에 소프트맥스를 씌우면 샘플마다 확률 3개가 됩니다.

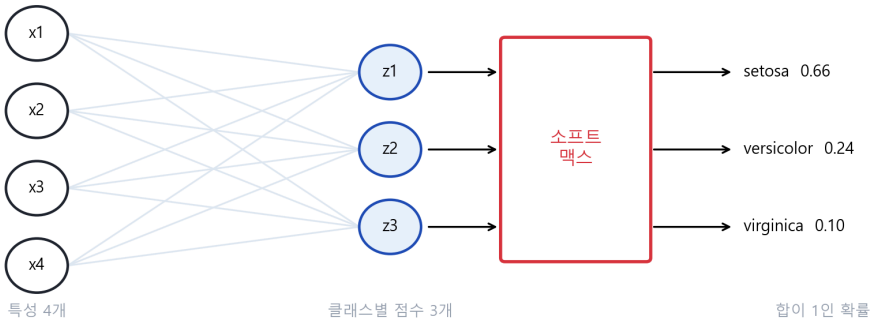


그림 3 소프트맥스 회귀의 구조. 특성 4개가 모든 클래스의 점수에 연결되고, 소프트맥스가 점수를 확률로 바꾼다

3) 손실 함수: 크로스 엔트로피

정답이 원-핫 벡터 [1, 0, 0]이면, 크로스 엔트로피는 **정답 칸의 확률에 -log를 씌운 값**만 남습니다. 정답 칸에 확률을 많이 줄수록 손실이 작고, 적게 줄수록 급격히 커집니다. 04장의 이진 크로스 엔트로피는 선택지가 2개인 특별한 경우입니다.

$$\text{cost} = -\sum_{j=1}^{k} y_j \log p_j = -\log p_c$$

y_j는 원-핫 정답(정답 칸 c만 1), p_j는 소프트맥스 확률. 0인 칸은 곱해서 사라지고 정답 칸 하나만 남습니다.

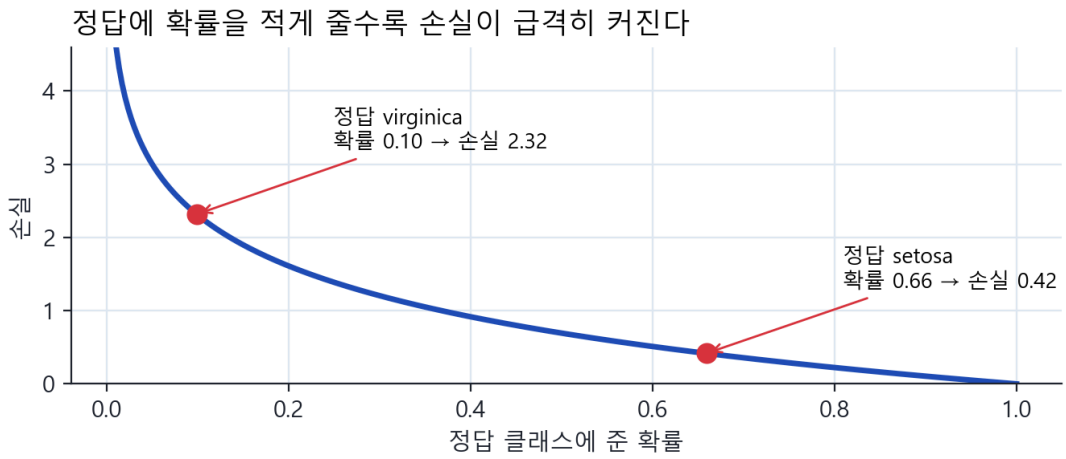


그림 4 실습 숫자로 본 크로스 엔트로피. 정답에 0.66을 주면 손실 0.42, 0.10만 주면 손실 2.32

비유

반장 선거 여론 조사

후보마다 지지 점수를 매긴 뒤, 전체 지지 중 각자의 몫을 퍼센트로 바꾸면 합이 100%가 됩니다(소프트맥스). 선거가 끝나고 실제 당선자에게 높은 퍼센트를 줬던 조사는 정확했던 것이고, 당선자에게 낮은 퍼센트를 줬다면 크게 틀린 조사입니다(크로스 엔트로피).

후보별 지지 점수	= 모델의 점수 z
지지율 (합 100%)	= 소프트맥스 확률
당선자에게 준 지지율로 채점	= 크로스 엔트로피

실습 2

소프트맥스와 크로스 엔트로피 손으로 계산하기

```python
import torch

# 붓꽃 한 송이에 대해 모델이 낸 품종별 점수 (아직 확률이 아님)
scores = torch.tensor([2.0, 1.0, 0.1])          # setosa, versicolor, virginica
exp = torch.exp(scores)
probs = exp / exp.sum()                          # 소프트맥스: e^점수 / 전체 합
print("점수      :", [round(s, 2) for s in scores.tolist()])
print("소프트맥스:", [round(p, 3) for p in probs.tolist()], " 합 =", round(probs.sum().item(), 3))
print("파이토치  :", [round(p, 3) for p in torch.softmax(scores, dim=0).tolist()])

# 크로스 엔트로피: 정답 칸의 확률에 -log를 씌운 값
for answer, name in [(0, "setosa"), (2, "virginica")]:
    loss = -torch.log(probs[answer])
    print(f"정답이 {name:10s}일 때 손실: {loss.item():.3f}  (정답 확률 {probs[answer].item():.3f})")
```

실행 결과

```
점수      : [2.0, 1.0, 0.1]
소프트맥스: [0.659, 0.242, 0.099]  합 = 1.0
파이토치  : [0.659, 0.242, 0.099]
정답이 setosa    일 때 손실: 0.417  (정답 확률 0.659)
정답이 virginica 일 때 손실: 2.317  (정답 확률 0.099)
```

직접 해 보기 점수를 [2.0, 1.0, 0.1]에서 [20, 10, 1]로 10배 키우면 확률은 어떻게 바뀌나요? 점수 차이가 클수록 소프트맥스가 한쪽으로 쏠린다는 것을 확인해 보세요.

05-02 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 소프트맥스를 거친 확률들의 합은 항상 1이다. (O / X) 이름 _____

2 소프트맥스는 점수의 순서를 뒤바꾼다. (O / X) 이름 _____

3 특성 4개, 클래스 3개일 때 가중치 W의 모양은? 이름 _____

① (3 × 4) ② (4 × 3) ③ (4 × 1) ④ (1 × 3)

4 크로스 엔트로피에서 결국 남는 값은? 이름 _____

① 정답 칸 확률의 -log ② 모든 확률의 합 ③ 가장 큰 점수

5 이진 크로스 엔트로피는 크로스 엔트로피에서 선택지가 _____개인 경우다. 이름 _____

6 실습에서 점수 2.0의 소프트맥스 확률은 _____였다. 이름 _____

7 반장 선거 여론 조사 비유로 소프트맥스와 크로스 엔트로피를 설명해 보세요. 이름 _____

원문: 05-02 소프트맥스 회귀 이해하기 — https://wikidocs.net/59427

05-03 소프트맥스 회귀 다양한 방법으로 구현하기

핵심 한 줄

크로스 엔트로피는 직접 쓰거나 F.cross_entropy 한 줄로 쓸 수 있고, F.cross_entropy는 소프트맥스까지 포함하므로 모델 출력(점수)을 그대로 넣는다.

왜 배우나

파이토치 코드에는 같은 손실을 계산하는 방법이 여러 가지 섞여 나옵니다. 네 가지가 모두 같은 값이라는 것을 확인해 두면 남의 코드를 읽을 때 헷갈리지 않습니다. 특히 "모델 끝에 소프트맥스를 넣지 않는다"는 규칙을 모르면 06장 이후 실습에서 자주 실수하게 됩니다.

1) 같은 손실, 네 가지 방법

방법	코드	입력
1) 직접	`(y_onehot * -torch.log(F.softmax(z))).sum(1).mean()`	점수, 원-핫 정답
2) log_softmax	`(y_onehot * -F.log_softmax(z)).sum(1).mean()`	점수, 원-핫 정답
3) nll_loss	`F.nll_loss(F.log_softmax(z), y)`	로그 확률, 정답 번호
4) cross_entropy	`F.cross_entropy(z, y)`	**점수 그대로**, 정답 번호

네 번째가 가장 짧고, 원-핫 벡터를 따로 만들 필요도 없습니다. 정답은 번호(0, 1, 2)로 주면 내부에서 알아서 처리합니다. 주의할 점은 `F.cross_entropy` 안에 소프트맥스가 이미 들어 있다는 것입니다. 모델 마지막에 소프트맥스를 또 넣으면 두 번 계산되어 학습이 이상해집니다.

2) 학습시키기

특성 4개, 샘플 8개, 클래스 3개(0, 1, 2)인 작은 데이터로 학습합니다. 모델은 `nn.Linear(4, 3)` 하나뿐입니다. 예측은 점수 3개 중 가장 큰 것의 위치(`argmax`)이고, 1000번 학습하면 8개를 모두 맞힙니다.



그림 5 실습과 같은 설정의 학습 곡선. 손실(파랑)이 줄어드는 동안 정확도(빨강)가 계단처럼 올라간다

비유

택배 보내기

택배를 보낼 때 포장부터 송장 쓰기까지 직접 할 수도 있고, 편의점에 물건만 맡기면 포장까지 해 주는 서비스를 쓸 수도 있습니다. F.cross_entropy는 포장(소프트맥스)까지 해 주는 서비스라서, 이미 포장한 상자를 또 맡기면 이중 포장이 됩니다.

직접 포장하고 보내기 = 방법 1~3 (소프트맥스 따로)
포장까지 해 주는 서비스 = F.cross_entropy
이중 포장 = 모델 끝 소프트맥스 + cross_entropy (실수)

실습 3 네 가지 손실 비교와 소프트맥스 회귀 학습

```
import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(1)
# 같은 손실을 계산하는 네 가지 방법 (위로 갈수록 직접, 아래로 갈수록 편하게)
z = torch.rand(3, 5)                                    # 샘플 3개, 클래스 5개
의 점수
y = torch.randint(5, (3,))                              # 정답 번호
y_onehot = F.one_hot(y, num_classes=5).float()
print("1) 직접 계산        :", round((y_onehot * -torch.log(F.softmax(z, dim
=1))).sum(dim=1).mean().item(), 4))
print("2) log_softmax 사용 :", round((y_onehot * -F.log_softmax(z, dim=1)).s
um(dim=1).mean().item(), 4))
print("3) nll_loss         :", round(F.nll_loss(F.log_softmax(z, dim=1),
y).item(), 4))
print("4) cross_entropy    :", round(F.cross_entropy(z, y).item(), 4), " ←
소프트맥스까지 포함")

# 특성 4개 → 클래스 3개(0, 1, 2) 분류
x_train = torch.FloatTensor([[1, 2, 1, 1], [2, 1, 3, 2], [3, 1, 3, 4], [4,
1, 5, 5],
                             [1, 7, 5, 5], [1, 2, 5, 6], [1, 6, 6, 6], [1,
7, 7, 7]])
y_train = torch.LongTensor([2, 2, 2, 1, 1, 1, 0, 0])

model = nn.Linear(4, 3)                         # 출력 3개 = 클래스별 점수
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
for epoch in range(1001):
    prediction = model(x_train)
    cost = F.cross_entropy(prediction, y_train)    # 소프트맥스 + 크로스 엔트
로피
    optimizer.zero_grad()
    cost.backward()
    optimizer.step()
    if epoch % 250 == 0:
        acc = (prediction.argmax(dim=1) == y_train).float().mean().item()
        print(f"Epoch {epoch:4d}  Cost: {cost.item():.4f}  정확도: {acc * 10
0:.1f}%")
print("예측:", model(x_train).argmax(dim=1).tolist())
print("정답:", y_train.tolist())
```

실행 결과

```
1) 직접 계산        : 1.4689
2) log_softmax 사용 : 1.4689
3) nll_loss         : 1.4689
4) cross_entropy    : 1.4689  ← 소프트맥스까지 포함
Epoch    0  Cost: 1.8495  정확도: 37.5%
Epoch  250  Cost: 0.5789  정확도: 75.0%
Epoch  500  Cost: 0.4519  정확도: 87.5%
Epoch  750  Cost: 0.3357  정확도: 87.5%
Epoch 1000  Cost: 0.2419  정확도: 100.0%
예측: [2, 2, 2, 1, 1, 1, 0, 0]
정답: [2, 2, 2, 1, 1, 1, 0, 0]
```

직접 해 보기 교재처럼 클래스로도 만들어 보세요: `__init__` 에 `self.linear = nn.Linear(4, 3)`, `forward` 에 `return self.linear(x)`. 결과가 같은가요?

확인 문제

05-03 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 F.cross_entropy 안에는 소프트맥스가 포함되어 있다. (O / X) 이름 _____

2 F.cross_entropy를 쓸 때는 모델 마지막에 소프트맥스를 꼭 넣어야 한다. (O / X) 이름 _____

3 F.cross_entropy에 정답을 주는 형태는? 이름 _____

① 정답 번호 (예: 2) ② 반드시 원-핫 벡터 ③ 확률

4 점수 중 가장 큰 것의 위치를 고르는 함수는? 이름 _____

① argmax ② softmax ③ mean

5 실습 모델 nn.Linear(4, 3)에서 3은 _____의 개수다. 이름 _____

6 실습에서 네 가지 방법의 손실 값은 모두 _____였다. 이름 _____

7 택배 비유로 "모델 끝에 소프트맥스를 넣지 않는 이유"를 설명해 보세요. 이름 _____

원문: 05-03 소프트맥스 회귀 다양한 방법으로 구현하기 — https://wikidocs.net/60575

05-04 소프트맥스 회귀로 MNIST 데이터 분류하기

핵심 한 줄

28×28 손글씨를 784칸으로 펴서 nn.Linear(784, 10) 하나로 숫자 10개의 점수를 내면, 은닉층 없이도 약 90% 이상을 맞힌다.

왜 배우나

지금까지 배운 것(미니 배치, 소프트맥스, 크로스 엔트로피)을 진짜 이미지 데이터에 처음 써 봅니다. 이 결과(약 91~92%)는 앞으로의 **기준 점수**가 됩니다. 06장의 다층 퍼셉트론(약 97%)과 08장의 CNN(약 98%)이 이 기준을 얼마나 넘는지 비교하면서, 층을 쌓는 효과를 숫자로 느낄 수 있습니다.

1) MNIST와 torchvision

MNIST는 0~9 손글씨 숫자 그림 7만 장(훈련 6만, 테스트 1만)입니다. 한 장은 28×28 흑백 그림이고, **torchvision**이라는 파이토치 도구 모음에서 한 줄로 내려받을 수 있습니다. `transforms.ToTensor()`는 그림을 0~1 사이 숫자의 텐서로 바꿔 줍니다.

2) 학습 흐름

1. DataLoader로 100장씩 묶어 꺼냅니다(03-04의 미니 배치).
2. (100, 1, 28, 28) 모양을 `view(-1, 784)`로 (100, 784)로 폅니다.
3. `nn.Linear(784, 10)`이 숫자별 점수 10개를 내고, `CrossEntropyLoss`가 소프트맥스와 손실을 계산합니다.
4. 학습이 끝나면 테스트 1만 장으로 정확도를 잽니다. 이때는 `torch.no_grad()`로 기울기 계산을 끕니다.

3) 모델이 배운 것 들여다보기

은닉층이 없으니 가중치 W(10 × 784)의 각 줄을 28×28로 다시 접으면, 숫자마다 "어느 칸이 칠해지면 점수를 올리고(빨강) 내리는지(파랑)"를 그대로 볼 수 있습니다. 0의 가중치에는 동그라미 모양이, 1에는 가운데 세로줄이 희미하게 보입니다. 모델이 숫자별 **템플릿**을 배운 셈입니다.

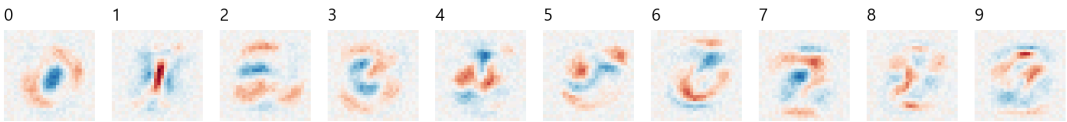


그림 6 1 에포크 학습한 소프트맥스 회귀의 숫자별 가중치. 빨강은 칠해지면 그 숫자 점수가 오르는 칸, 파랑은 내려가는 칸

비유

숫자별 투명 도안 겹쳐 보기

숫자 0~9의 투명 도안 10장을 가지고, 새 손글씨 위에 하나씩 겹쳐 보며 가장 잘 맞는 도안을 고르는 방식입니다. 간단하지만 꽤 잘 맞힙니다. 다만 글씨가 기울거나 옆으로 치우치면 도안과 어긋나서 틀리기 쉽습니다. 이 약점을 06장의 은닉층과 08장의 CNN이 보완합니다.

투명 도안 10장	= 가중치 W의 10줄
겹쳐서 맞는 정도	= 숫자별 점수 (Wx + b)
가장 잘 맞는 도안 고르기	= argmax

실습 4

MNIST 소프트맥스 회귀

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
import torchvision.datasets as dsets
import torchvision.transforms as transforms

torch.manual_seed(777)
train = dsets.MNIST(root="MNIST_data/", train=True, transform=transforms.To
Tensor(), download=True)
test = dsets.MNIST(root="MNIST_data/", train=False, transform=transforms.To
Tensor(), download=True)
loader = DataLoader(train, batch_size=100, shuffle=True, drop_last=True)
print("그림 한 장의 모양:", tuple(train[0][0].shape), " 정답:", train[0][1])

linear = nn.Linear(784, 10)                # 784칸 → 숫자 10개의 점수 (은닉층
없음)
loss_fn = nn.CrossEntropyLoss()            # 내부에서 소프트맥스까지 계산
optimizer = torch.optim.SGD(linear.parameters(), lr=0.1)

for epoch in range(3):                     # 교재는 15번, 여기서는 3번만
    for x, y in loader:
        x = x.view(-1, 28 * 28)            # (100, 1, 28, 28) → (100, 784)
        loss = loss_fn(linear(x), y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    print(f"Epoch {epoch + 1}  마지막 배치 Loss: {loss.item():.4f}")

with torch.no_grad():
    x_test = test.data.view(-1, 784).float() / 255
    pred = linear(x_test).argmax(dim=1)
    print("테스트 정확도:", round((pred == test.targets).float().mean().item
(), 4))
    print("첫 5장 예측:", pred[:5].tolist(), " 정답:", test.targets[:5].tolis
t())
```

실행 결과

```
그림 한 장의 모양: (1, 28, 28)  정답: 5
Epoch 1  마지막 배치 Loss: 0.4472
Epoch 2  마지막 배치 Loss: 0.3161
Epoch 3  마지막 배치 Loss: 0.2932
테스트 정확도: 0.9161
첫 5장 예측: [7, 2, 1, 0, 4]  정답: [7, 2, 1, 0, 4]
```

직접 해 보기 에포크를 3에서 15로 늘리면(교재 설정) 정확도가 얼마나 오를까요? 코랩 CPU에서 에포크당 약 10초 정도 걸립니다.

확인 문제

05-04 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 실습 모델에는 은닉층이 없다. (O / X) 이름 _____

2 MNIST 그림 한 장은 784칸으로 펴서 모델에 넣었다. (O / X) 이름 _____

3 그림을 0~1 사이 숫자의 텐서로 바꿔 주는 것은? 이름 _____

① transforms.ToTensor() ② nn.Linear ③ argmax

4 실습 모델의 모양으로 맞는 것은? 이름 _____

① nn.Linear(784, 10) ② nn.Linear(10, 784) ③ nn.Linear(28, 28)

5 MNIST를 쉽게 내려받게 해 주는 파이토치 도구 모음은 _____이다. 이름 _____

6 실습의 테스트 정확도는 약 _____%였다. 이름 _____

7 투명 도안 비유로 소프트맥스 회귀가 숫자를 알아보는 방식을 설명해 보세요. 이름 _____

원문: 05-04 소프트맥스 회귀로 MNIST 데이터 분류하기 — https://wikidocs.net/60324

오늘의 요약

다시 보기



핵심 한 줄 모음

소챕터	핵심 한 줄
05-01	정답 자리만 1인 원-핫 벡터로 선택지를 표현하면, 원래 없던 순서와 거리가 생기지 않는다.
05-02	선택지별 점수를 소프트맥스로 확률로 바꾸고, 크로스 엔트로피로 정답 확률이 커지게 학습한다.
05-03	F.cross_entropy는 소프트맥스를 포함하므로 모델 출력(점수)과 정답 번호를 그대로 넣는다.
05-04	nn.Linear(784, 10) 하나로 MNIST를 약 91% 맞힌다. 앞으로의 기준 점수다.

다음

다음 자료 예고: 06장 인공 신경망 (10/7)

오늘 만든 모델은 모두 "곱하고 더하고, 한 번 변환"하는 한 층짜리입니다. 내일은 이 층을 여러 개 쌓은 인공 신경망을 배우고, MNIST 정확도가 91%에서 97%로 오르는 것을 확인합니다.

정답과 해설

05-01 원-핫 인코딩

1. O
2. X — 모든 쌍의 거리가 같아서 비슷함을 표현하지 못합니다. 12장의 워드 임베딩이 이를 해결합니다.
3. ③
4. ①
5. 0, 1, 0
6. 1.414
7. 예시 답: 후보마다 칸이 있고 한 칸에만 도장을 찍듯, 정답 자리만 1로 표시한다. 기호 번호 사이의 차이에는 의미가 없다.

05-02 소프트맥스 회귀 이해하기

1. O
2. X — 점수가 큰 순서가 확률이 큰 순서와 같습니다.
3. ② — (특성 수 × 클래스 수)
4. ①
5. 2
6. 0.659
7. 예시 답: 후보별 지지 점수를 합 100%인 지지율로 바꾸는 것이 소프트맥스, 실제 당선자에게 높은 지지율을 줬는지로 조사를 채점하는 것이 크로스 엔트로피다.

05-03 다양한 방법으로 구현하기

1. O
2. X — 넣으면 소프트맥스가 두 번 계산됩니다.
3. ①
4. ①
5. 클래스(선택지)
6. 1.4689
7. 예시 답: F.cross_entropy는 포장(소프트맥스)까지 해 주는 택배 서비스라서, 직접 포장한 상자를 또 맡기면 이중 포장이 된다.

05-04 MNIST 분류하기

1. O
2. O
3. ①
4. ①
5. torchvision
6. 91.6 (실행 결과 0.9161)

7. 예시 답: 숫자별 투명 도안 10장을 손글씨 위에 겹쳐 보고 가장 잘 맞는 것을 고르는 방식이다. 도안이 가중치, 맞는 정도가 점수다.

용어 정리

용어	쉬운 뜻
다중 클래스 분류	셋 이상의 선택지 중 하나를 고르는 문제
원-핫 인코딩	선택지 수만큼 칸을 만들고 정답 자리만 1로 표시하는 방법
소프트맥스 함수	점수들을 합이 1인 확률로 바꾸는 함수
소프트맥스 회귀	선택지별 점수(Wx + b)에 소프트맥스를 씌워 다중 분류를 하는 모델
크로스 엔트로피	정답 칸의 확률에 -log를 씌운 손실. 정답 확률이 작을수록 커진다
F.cross_entropy	소프트맥스 + 크로스 엔트로피를 한 번에 계산하는 파이토치 함수
argmax	가장 큰 값의 위치(번호)를 돌려주는 함수
torchvision	MNIST 같은 이미지 데이터와 변환 도구를 모아 둔 파이토치 패키지